Build: Growth Engine

Objective

Create a deterministic **Growth Engine** that answers:

“Can this business scale and win?”

It reads:

- business profile
- market profile
- positioning profile
- channels
- competitors
- pricing power
- growth signals

It writes structured output for:

- Master Aggregator
- Fragility Index
- Scenario Simulation
- Adaptive Scenario Intelligence
- AI Narratives
- PDF Reports

Recommended Folder

`/src/lib/engines/growth`

Suggested structure:

```
/src/lib/engines/growth
 ├── types.ts
```

```
├── rules.ts
├── growth-engine.ts
├── growth-engine.test.ts
└── index.ts
```

1. Input Contract

```
export interface GrowthEngineInput {
  businessId: string;

  productDelivery: {
    deliveryModel: "manual" | "hybrid" | "automated";
    marginalCostTrend: "increasing" | "stable" | "decreasing";
    digitalComponentPct?: number;
  };

  marketExpansion: {
    currentMarketScope: "local" | "regional" | "national" | "global";
    expansionChannelsAvailable?: number;
    geographicScalability: boolean;
    regulatoryBarriers?: "low" | "medium" | "high";
  };

  pricing: {
    priceElasticityIndicator: "low" | "medium" | "high";
    differentiationLevel: "low" | "medium" | "high";
    grossMarginTrend?: "increasing" | "stable" | "declining";
  };

  competitive: {
    numberOfDirectCompetitors?: number;
    marketPosition: "leader" | "challenger" | "niche" | "unknown";
    switchingCostLevel?: "low" | "medium" | "high";
    brandStrength?: "low" | "medium" | "high";
  };

  metadata?: {
```

```
    source?: "manual" | "imported" | "integration";
    confidenceLevel?: "LOW" | "MEDIUM" | "HIGH";
    rulesetVersion?: string;
  };
}
```

2. Output Contract

```
export interface GrowthEngineOutput {
  engine: "growth";
  version: string;
 businessId: string;

  metrics: {
    deliveryModel: string;
    marginalCostTrend: string;
    digitalComponentPct: number | null;
    currentMarketScope: string;
    expansionChannelsAvailable: number;
    geographicScalability: boolean;
    regulatoryBarriers: string | null;
    priceElasticityIndicator: string;
    differentiationLevel: string;
    grossMarginTrend: string | null;
    marketPosition: string;
    switchingCostLevel: string | null;
    brandStrength: string | null;
    numberOfDirectCompetitors: number | null;
  };

  scores: {
    scalabilityScore: number;
    expansionScore: number;
    pricingScore: number;
    competitiveScore: number;
    growthScore: number;
  };
```

```
classification: {
  scenario:
    | "HIGH_GROWTH_POTENTIAL"
    | "GROWTH_READY"
    | "LIMITED_GROWTH"
    | "LOW_SCALABILITY"
    | "SCALING_CONSTRAINT"
    | "COMMODITY_TRAP";
  label: string;
};

riskFlags: Array<{
  code: string;
  severity: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL";
  explanation: string;
  recommendedAction: string;
  priorityScore: number;
}>;

recommendations: Array<{
  type: string;
  priority: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL";
  text: string;
  expectedImpact?: string;
}>;

interpretation: {
  headline: string;
  driverSummary: string;
  businessInterpretation: string;
  riskOutlook: string;
  recommendedActions: string[];
};

confidence: {
  level: "LOW" | "MEDIUM" | "HIGH";
  note: string;
  penaltyApplied: boolean;
```

```
  };

  trace: Record<string, unknown>;
}
```

3. Normalisation Rules

Percentage fields

If `digitalComponentPct` is passed as `60`, normalise to `0.6`.

If passed as `0.6`, keep as `0.6`.

```
function normalizePct(value?: number): number | null {
  if (value === undefined || value === null) return null;
  return value > 1 ? value / 100 : value;
}
```

4. Scoring Rules

A. Scalability Potential Score

```
if deliveryModel === "automated" && marginalCostTrend ===
"decreasing":
  scalabilityScore = 100

else if deliveryModel === "hybrid" && marginalCostTrend === "stable":
  scalabilityScore = 70

else if deliveryModel === "manual" && marginalCostTrend ===
"increasing":
  scalabilityScore = 30

else if deliveryModel === "manual":
```

```
  scalabilityScore = 10

else:
  scalabilityScore = 40
```

Adjustment:

```
if digitalComponentPct !== null && digitalComponentPct > 0.6:
  scalabilityScore += 10
  cap at 100
```

B. Market Expansion Readiness Score

```
if currentMarketScope === "global" && geographicScalability === true
&& regulatoryBarriers !== "high":
  expansionScore = 100

else if currentMarketScope === "national" && geographicScalability ===
true:
  expansionScore = 70

else if currentMarketScope === "regional":
  expansionScore = 40

else if currentMarketScope === "local" && regulatoryBarriers ===
"high":
  expansionScore = 10

else:
  expansionScore = 40
```

Adjustment:

```
if expansionChannelsAvailable >= 3:
  expansionScore += 10
  cap at 100
```

Penalty:

```
if regulatoryBarriers === "high":
  expansionScore -= 10
  floor at 0
```

C. Pricing Power Score

```
if differentiationLevel === "high" && priceElasticityIndicator ===
"low":
  pricingScore = 100

else if differentiationLevel === "medium":
  pricingScore = 70

else if differentiationLevel === "low":
  pricingScore = 40

else:
  pricingScore = 10
```

Commodity case:

```
if differentiationLevel === "low" && priceElasticityIndicator ===
"high":
  pricingScore = 10
```

Adjustment:

```
if grossMarginTrend === "increasing":
  pricingScore += 10
  cap at 100
```

Penalty:

```
if grossMarginTrend === "declining":
  pricingScore -= 10
  floor at 0
```

D. Competitive Position Score

```
if marketPosition === "leader" || marketPosition === "niche":
  competitiveScore = 100

else if marketPosition === "challenger":
  competitiveScore = 70

else if marketPosition === "unknown":
  competitiveScore = 10

else:
  competitiveScore = 40
```

Adjustment:

```
if switchingCostLevel === "high":
  competitiveScore += 10
  cap at 100
```

Penalty:

```
if brandStrength === "low":
  competitiveScore -= 10
  floor at 0
```

Optional competitor penalty:

```
if numberOfDirectCompetitors !== undefined &&
numberOfDirectCompetitors > 20:
  competitiveScore -= 10
  floor at 0
```

5. Final Growth Score

```
growth_score =
0.30 × scalability_score +
0.25 × expansion_score +
```

```
0.25 × pricing_score +
0.20 × competitive_score
```

6. Classification Logic

Override scenarios come first.

```
if scalabilityScore <= 30:
  SCALING_CONSTRAINT

else if pricingScore <= 30 && competitiveScore <= 40:
  COMMODITY_TRAP

else if growthScore >= 80:
  HIGH_GROWTH_POTENTIAL

else if growthScore >= 60:
  GROWTH_READY

else if growthScore >= 40:
  LIMITED_GROWTH

else:
  LOW_SCALABILITY
```

Labels:

```
HIGH_GROWTH_POTENTIAL = "High Growth Potential"
GROWTH_READY = "Growth Ready"
LIMITED_GROWTH = "Limited Growth"
LOW_SCALABILITY = "Low Scalability"
SCALING_CONSTRAINT = "Scaling Constraint"
COMMODITY_TRAP = "Commodity Trap"
```

7. Risk Flags

LOW_SCALABILITY

Trigger:

```
scalabilityScore <= 30
```

Severity:

```
HIGH
```

Action:

```
Redesign delivery model, increase automation, or reduce manual
delivery dependency.
```

EXPANSION_LIMITED

Trigger:

```
expansionScore <= 40
```

Severity:

```
MEDIUM
```

Action:

```
Develop scalable channels, reduce regional dependency, or remove
expansion barriers.
```

WEAK_PRICING_POWER

Trigger:

```
pricingScore <= 40
```

Severity:

HIGH

Action:

Increase differentiation, strengthen positioning, or repackage the offer.

COMPETITIVE_PRESSURE

Trigger:

```
competitiveScore <= 40
```

Severity:

HIGH

Action:

Strengthen market positioning, improve brand trust, or build switching costs.

COMMODITY_RISK

Trigger:

```
differentiationLevel === "low" && priceElasticityIndicator === "high"
```

Severity:

CRITICAL

Action:

Avoid competing mainly on price. Create stronger differentiation or target a more specific customer segment.

HIGH_REGULATORY_BARRIER

Trigger:

```
regulatoryBarriers === "high"
```

Severity:

```
MEDIUM
```

Action:

```
Clarify compliance requirements before expanding into new markets.
```

WEAK_BRAND_STRENGTH

Trigger:

```
brandStrength === "low"
```

Severity:

```
MEDIUM
```

Action:

```
Improve credibility, social proof, case studies, reviews, and
authority positioning.
```

8. Recommendations Logic

Generate top 3 recommendations based on risk priority.

Priority order:

1. COMMODITY_RISK
2. LOW_SCALABILITY
3. WEAK_PRICING_POWER

4. COMPETITIVE_PRESSURE
5. EXPANSION_LIMITED
6. HIGH_REGULATORY_BARRIER
7. WEAK_BRAND_STRENGTH

Example recommendations:

```
[
  {
    type: "differentiation",
    priority: "CRITICAL",
    text: "Strengthen differentiation to avoid competing mainly on
price."
  },
  {
    type: "scalability",
    priority: "HIGH",
    text: "Increase automation or redesign delivery to reduce manual
scaling constraints."
  },
  {
    type: "market_expansion",
    priority: "MEDIUM",
    text: "Develop at least three scalable acquisition channels before
national expansion."
  }
]
```

9. Interpretation Rules

HIGH_GROWTH_POTENTIAL

The business shows strong potential to scale and expand.

GROWTH_READY

The business is growth-ready but should optimise specific constraints before scaling aggressively.

LIMITED_GROWTH

The business has limited growth potential under its current structure.

LOW_SCALABILITY

The business is difficult to scale due to weak leverage, limited expansion potential, or poor positioning.

SCALING_CONSTRAINT

The business has structural constraints that may prevent it from scaling efficiently.

COMMODITY_TRAP

The business is exposed to commodity pressure because differentiation and pricing power are weak.

10. Confidence Logic

Use this order:

```
if metadata.confidenceLevel exists:
  use it
else if metadata.source === "integration":
  HIGH
else if marketPosition !== "unknown" && differentiationLevel &&
priceElasticityIndicator:
  MEDIUM
```

```
else:
  LOW
```

Penalty applied if:

```
marketPosition === "unknown"
```

Confidence notes:

```
HIGH: Based on verified market or integrated business data.
MEDIUM: Based on structured market and positioning inputs.
LOW: Based mainly on assumptions and should be validated with market
evidence.
```

11. Trace Object

Include:

```
trace: {
  formulas: {
    growthScore:
      "0.30 * scalabilityScore + 0.25 * expansionScore + 0.25 *
pricingScore + 0.20 * competitiveScore"
  },
  inputsUsed: input,
  scoringRules: {
    scalabilityWeight: 0.3,
    expansionWeight: 0.25,
    pricingWeight: 0.25,
    competitiveWeight: 0.2
  }
}
```

12. Unit Tests

Create tests for:

High Growth Potential Case

Input:

- automated delivery
- decreasing marginal cost
- global or national market
- high differentiation
- low price sensitivity
- strong niche/leader

Expected:

- HIGH_GROWTH_POTENTIAL
- high growth score

Scaling Constraint Case

Input:

- manual delivery
- increasing marginal cost

Expected:

- SCALING_CONSTRAINT
- LOW_SCALABILITY risk

Commodity Trap Case

Input:

- low differentiation
- high price elasticity
- weak/unknown market position

Expected:

- COMMODITY_TRAP
- COMMODITY_RISK
- WEAK_PRICING_POWER

Expansion Limited Case

Input:

- local scope
- high regulatory barriers

Expected:

- EXPANSION_LIMITED
- HIGH_REGULATORY_BARRIER

Weak Brand Case

Input:

- brandStrength low

Expected:

- WEAK_BRAND_STRENGTH

Digital Bonus Case

Input:

- digitalComponentPct 70 or 0.7

Expected:

- scalability bonus applied safely

13. Acceptance Criteria

Codex should complete this when:

- Growth Engine accepts the defined input contract
- It returns the defined output contract
- All scoring rules work
- Override classifications work
- Risk flags are generated correctly
- Recommendations are prioritised
- Interpretation is deterministic
- Confidence is included
- Trace is included
- Unit tests pass
- Percentage normalisation works
- No database calls inside engine
- No LLM calls inside engine

14. Codex Prompt

```
Build the EnterprateAI Growth Engine in /src/lib/engines/growth.

The engine must be a pure deterministic TypeScript module. It must not
call the database, API, or LLM.

It should accept GrowthEngineInput and return GrowthEngineOutput.

Implement:
- percentage normalisation
- scalability score
- market expansion score
- pricing power score
```

- competitive position score
- final growth score
- override scenario classification
- risk flags
- top 3 recommendations
- interpretation
- confidence layer
- trace object

Use the shared computation library from /src/lib/computation for weighted score if available.

Create:
- types.ts
- rules.ts
- growth-engine.ts
- growth-engine.test.ts
- index.ts

Add unit tests for:
- high growth potential case
- scaling constraint case
- commodity trap case
- expansion limited case
- weak brand case
- digital bonus case

Export the engine from index.ts.